

Building a REST API Server with PHP

| Part 2: PDO/MySQL Storage

From JSON to MySQL

In the previous lecture we built a complete student API using a JSON file for storage.

In this lecture we swap out only the storage layer — the URLs, HTTP methods, router, and even the test page stay exactly the same.

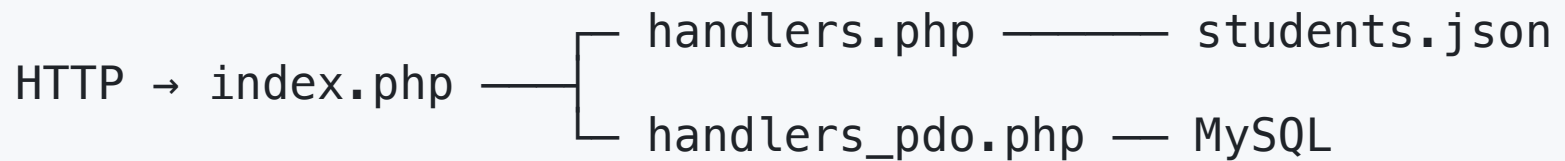
Why Move Away from JSON Files?

- Every save rewrites the **entire** file, even to change one student
- No protection against two requests writing at the same time
- Filtering or searching means loading everything into memory first
- `max_id + 1` can reuse an ID after a delete (we saw this earlier)

A real database solves all four with proper storage, indexes, and transactions — which is exactly what PDO gives us access to.

Bridge to Project 1: Replace JSON with PDO

The REST contract does not depend on the storage technology.



IMPORTANT

- The URLs, methods, router, frontend, and tests stay the same — only the handler implementation changes (the Week 3 PDO pattern, behind the REST handlers).
- "Same URLs" isn't enough: the PDO handlers also match the same status codes, response shape (`success`, `message`, `data`), and value types (e.g. `year` as an integer) as the JSON version.

Three Steps for the Transition

| Step 1: MySQL Service & Automation

The Week 6 schema evolves the Week 3 `students` table to match the API model: `name`, `email`, `major`, and `year`.

`./start_server.sh --pdo` is fully automated:

1. **Auto-starts MySQL:** If MySQL is not running, the script starts it automatically (`brew services` on macOS, `service` / `systemctl` on WSL2/Linux).
2. **Auto-creates Database & Table:** Automatically sets up `studentdb` and table `students` from `schema.sql`.
3. **Auto-resets on Startup:** Clears old rows so every test run starts from a clean slate (pass `--no-reset` to keep data).

However, you can start the service manually if needed

```
brew services start mysql # (macOS)
sudo service mysql start # (WSL2/Ubuntu)

# Start the php
STORAGE="pdo"
STUDENT_STORAGE="$STORAGE" exec php -S localhost:8080
```

Step 2: The PDO Connection

The database.php makes this connection.

```
function db(): PDO
{
    static $pdo = null;
    if ($pdo instanceof PDO) {
        return $pdo;
    }

    $host = getenv('DB_HOST') ?: 'localhost';
    $port = getenv('DB_PORT') ?: '3306';
    $name = getenv('DB_NAME') ?: 'studentdb';
    $user = getenv('DB_USER') ?: 'ase230';
    $password = getenv('DB_PASSWORD') !== false ? getenv('DB_PASSWORD') : 'ase230pass';
    $dsn = "mysql:host={$host};port={$port};dbname={$name};charset=utf8mb4";

    $pdo = new PDO($dsn, $user, $password, [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES => false,
    ]);
    return $pdo;
}
```

- `static $pdo` — connect once per request, reuse the same connection afterward.
- `ATTR_ERRMODE => ERRMODE_EXCEPTION` — a bad query throws instead of failing silently.
- `ATTR_DEFAULT_FETCH_MODE => FETCH_ASSOC` — rows come back as associative arrays, matching the JSON version's shape.
- `ATTR_EMULATE_PREPARES => false` — uses real server-side prepared statements, not PHP-emulated ones.

Step 3: Select the PDO Handler

`index.php` chooses a handler without changing its routing code.

```
$storage = getenv('STUDENT_STORAGE') ?: 'json';  
require_once $storage === 'pdo'  
    ? __DIR__ . '/handlers_pdo.php'  
    : __DIR__ . '/handlers.php';
```

- Default: uses `handlers.php` (JSON storage).
- Switch to MySQL: pass `STUDENT_STORAGE=pdo` in the shell or run `./start_server.sh --pdo`.

CRUD in SQL

Create

```
function create_student(): void
{
    $input = get_pdo_request_data();

    // Same POST contract as handlers.php: only name/email are required,
    // major/year default like the JSON version does.
    if (!array_key_exists('name', $input) || !array_key_exists('email', $input)) {
        send_pdo_error('name and email are required', 400);
        return;
    }
    $input['major'] = $input['major'] ?? '';
    $input['year'] = $input['year'] ?? 1;

    $error = validate_pdo_student($input, false);
    if ($error !== null) {
        send_pdo_error($error, 400);
        return;
    }
}
```

Create (Insert, Read Back, Respond)

```
$statement = db()->prepare(
    'INSERT INTO students (name, email, major, year) VALUES (:name, :email, :major, :year)'
);
$statement->execute([
    ':name' => trim($input['name']),
    ':email' => trim($input['email']),
    ':major' => trim($input['major']),
    ':year' => (int)$input['year'],
]);

$id = (int)db()->lastInsertId();
$statement = db()->prepare(
    'SELECT id, name, email, major, year, created_at, updated_at FROM students WHERE id = :id'
);
$statement->execute([':id' => $id]);
$student = $statement->fetch();

http_response_code(201);
echo json_encode([
    'success' => true,
    'message' => 'Student created successfully',
    'data' => $student,
]);
}
```

`lastInsertId()` gives the new row's id; we
`SELECT` it back so the response reflects what the
database actually stored.

Read All

```
function get_all_students(): void
{
    $students = db()->query(
        'SELECT id, name, email, major, year, created_at, updated_at FROM students ORDER BY id'
    )->fetchAll();
    echo json_encode(['success' => true, 'data' => $students, 'count' => count($students)]);
}
```

Same response shape as `get_all_students()` in `handlers.php` — `success`, `data`, `count`.

IMPORTANT

Read One

```
function get_student(int $id): void
{
    $statement = db()->prepare(
        'SELECT id, name, email, major, year, created_at, updated_at FROM students WHERE id = :id'
    );
    $statement->execute([':id' => $id]);
    $student = $statement->fetch();

    if (!$student) {
        send_pdo_error('Student not found', 404);
        return;
    }
    echo json_encode(['success' => true, 'data' => $student]);
}
```

:id is a named placeholder — PDO substitutes the bound value safely, so that value can never be interpreted as SQL syntax.

| Update

PUT requires all four fields
(`validate_pdo_student(..., true)`), so the SQL
always updates the same four columns:

```
UPDATE students
SET name = :name, email = :email, major = :major, year = :year
WHERE id = :id
```

There is no need to build the **SET** part dynamically
because **PUT** replaces the whole student record, so
every column has a new value.

```

function update_student(int $id): void
{
    $input = get_pdo_request_data();
    $error = validate_pdo_student($input, true);
    if ($error !== null) {
        send_pdo_error($error, 400);
        return;
    }

    $statement = db()->prepare(
        'UPDATE students SET name = :name, email = :email, major = :major, year = :year, ' .
        'updated_at = CURRENT_TIMESTAMP WHERE id = :id'
    );
    $statement->execute([
        ':id' => $id,
        ':name' => trim($input['name']),
        ':email' => trim($input['email']),
        ':major' => trim($input['major']),
        ':year' => (int)$input['year'],
    ]);
}

```

Update (Read Back and Respond)

```
$statement = db()->prepare(
    'SELECT id, name, email, major, year, created_at, updated_at FROM students WHERE id = :id'
);
$statement->execute([':id' => $id]);
$student = $statement->fetch();
if (!$student) {
    send_pdo_error('Student not found', 404);
    return;
}

echo json_encode([
    'success' => true,
    'message' => 'Student updated successfully',
    'data' => $student,
]);
}
```

Delete

```
function delete_student(int $id): void
{
    $statement = db()->prepare(
        'SELECT id, name, email, major, year, created_at, updated_at FROM students WHERE id = :id'
    );
    $statement->execute([':id' => $id]);
    $student = $statement->fetch();
    if (!$student) {
        send_pdo_error('Student not found', 404);
        return;
    }

    db()->prepare('DELETE FROM students WHERE id = :id')->execute([':id' => $id]);
    echo json_encode([
        'success' => true,
        'message' => 'Student deleted successfully',
        'data' => $student,
    ]);
}
```

We **SELECT** the row before deleting it, so the response can still show what was removed.

Keeping the Same Contract

```
function validate_pdo_student(array $input, bool $allRequired): ?string
{
    // Same rules as validate_student_fields() in handlers.php – this is
    // why the JSON and MySQL versions of the API accept and reject the
    // same input.
    foreach (['name', 'email', 'major', 'year'] as $field) {
        if ($allRequired && !array_key_exists($field, $input)) {
            return "{$field} is required";
        }
    }
    if (array_key_exists('email', $input) && !filter_var($input['email'], FILTER_VALIDATE_EMAIL)) {
        return 'email is not a valid email address';
    }
    // ...and matching checks for name, major, and year (see handlers_pdo.php)
    return null;
}
```

- `$allRequired` tells the validation function which operation is being checked.
- For `create_student()`, it is `false`, so only `name` and `email` are required, and `major`/`year` can use default values.
- For `update_student()`, it is `true`, so all fields are required because `PUT` replaces the whole resource.

Run with MySQL/PDO

Start the server using the automation script:

```
./start_server.sh --pdo
```

- Automatically starts MySQL service if it is not running
- Automatically creates `studentdb` schema and resets data

- Sets `STUDENT_STORAGE=pdo` and loads `handlers_pdo.php`
- To preserve existing database records across restarts:

```
./start_server.sh --pdo --no-reset
```

Running Directly with PHP (Alternative)

If starting the PHP development server manually:

macOS / Linux:

```
STUDENT_STORAGE=pdo \  
DB_USER=ase230 \  
DB_PASSWORD=ase230pass \  
php -S localhost:8080
```

Windows PowerShell:

```
$env:STUDENT_STORAGE="pdo"  
$env:DB_USER="ase230"  
$env:DB_PASSWORD="ase230pass"  
php -S localhost:8080
```

Resetting Data Between Test Runs

To reset storage without restarting the server:

```
# Reset MySQL (and JSON storage) back to an empty state:
./reset_db.sh --pdo          # or: ./reset_db.sh --mysql

# Reset JSON storage only:
./reset_db.sh
```

- Drops and recreates `studentdb` from `schema.sql`
- Re-initializes `data/students.json` to an empty array `[]`
- Guarantees tests and UI start from clean 0 rows

| Verify: Browser Test Page Access

Open the visual test page **via HTTP**:

```
http://localhost:8080/student_api_test.html
```

WARNING: Never open

student_api_test.html directly as a local file (**file:///...**)!

Double-clicking the HTML file opens it under the **file://** protocol. Browser security (CORS) treats it as an untrusted origin and blocks all fetch requests to

http://localhost:8080. You **must** access it through the HTTP server URL above.

The HTML page covers success cases; verify failure cases with cURL.

Stopping the Server and MySQL

When finished testing, safely shut down services:

```
# Stop both PHP server and MySQL service:
./stop_server.sh

# Or stop individually:
./stop_server.sh --php      # stop PHP development server only
./stop_server.sh --mysql    # stop MySQL service only
```

- Compatible with macOS (Homebrew) and Linux / WSL2 (`service` / `systemctl`)
- Frees port 8080 and releases system resources cleanly

Coming Next

The Testing lecture covers designing a success-and-failure test suite for this API; CORS was already covered in the JSON lecture, since it applies to either storage backend equally.